

# Elyxion Research: Whop is recording every sale twice, and about a quarter of page views twice

Prepared 9 September 2026 for the Elyxion Research development team. Everything below was measured against the live site and the live Whop events API. Where a number appears, it came off the wire rather than out of the source code.

The tracking you installed **is working**. The pixel loads, events reach Whop, and Whop accepts and stores them. Nothing here says start over. There are two duplication bugs and one smaller hygiene issue, and all three are small fixes. Both bugs are of the kind "fires more than once", never "does not fire", which is a much better position to be in.

## The problems, one line each

1. **Every sale is recorded twice**, because the purchase is sent under two different event ids. Revenue in Whop currently reads about double the truth.
2. **About 23% of page views are recorded twice**, because the route tracker fires more than once for a single navigation.
3. **Your own admin browsing is being tracked as customer traffic**.

## 1. Every sale counts twice

### The evidence

Two real orders came through on 8 September 2026. Whop holds **four** purchase events for them:

| Order        | Event id as recorded  | Time (UTC) | Value     |
|--------------|-----------------------|------------|-----------|
| ELX-MTT0P57O | ELX-MTT0P57O          | 18:43:25   | \$193.465 |
| ELX-MTT0P57O | purchase_ELX-MTT0P57O | 18:43:26   | \$193.465 |
| ELX-MTT4OENS | ELX-MTT4OENS          | 20:34:41   | \$269.955 |
| ELX-MTT4OENS | purchase_ELX-MTT4OENS | 20:34:42   | \$269.955 |

Each pair is the same sale: same customer id, same email address, same order value, same checkout URL, one second apart.

**\$463.42 of real orders is being reported as \$926.84.**

### Why Whop cannot collapse them

Whop deduplicates on `event_id`. Two events carrying the same `event_id` are treated as one sale no matter how many times they arrive. That is exactly the mechanism that is supposed to make a browser send and a server send safe to run alongside each other.

Here the two senders agree on everything about the sale **except the format of the id**:

- one sends the order number on its own, `ELX-MTT4OENS`

- the other sends it with a prefix, `purchase_ELX-MTT40ENS`

Because the ids differ, dedupe never engages and both are stored. Each sender is individually well behaved. It is the mismatch *between* them that causes this, which is why it is invisible when either one is reviewed on its own.

## The fix

**Make both senders emit a byte-identical `event_id` for the same order.**

The browser side currently sends the bare order number. In the order confirmation component the call is shaped like this:

```
whop.track("whop_purchase", orderId, {
  value,
  currency: "USD",
  event_id: orderId,
  email,
  phone
});
```

Change that one field so it matches the prefixed form the other sender already uses:

```
event_id: "purchase_" + orderId,
```

Standardizing on the prefixed form is the recommendation, because it is the convention the server side is already emitting, so it is a one line change on one side rather than a change on both. If you would rather standardize on the bare form instead, that is equally correct **provided both senders change together**. What must not happen is one of each.

## How to verify it worked

After deploying, place one order and look at the Whop events view for that order number. You should see **exactly one** `whop_purchase` event, not two.

Do not judge this by whether the request succeeded. Both of today's duplicates return success. The only meaningful check is the **count of stored events for a single order**.

## Please note this is not retroactive

The duplicates already recorded stay recorded. Whop will not go back and merge them. For any revenue reporting covering 8 September 2026 onward, purchase count and purchase value in that window read roughly double until the fix ships.

## 2. About a quarter of page views are duplicates

---

### The evidence

Across the last ten days there are **407** page events, of which **93 are a redundant second copy** of a page view already recorded for the same visitor on the same path. That is about **23%**. Of those 93, **80 land in the same second** as the original and 13 land one to two seconds later.

It affects most of the site, not one route:

| Path              | Duplicate pairs |
|-------------------|-----------------|
| /home             | 34              |
| /peptides         | 12              |
| /cart             | 10              |
| /                 | 9               |
| /checkout         | 6               |
| /peptides/rt-glp3 | 4               |
| /admin/orders     | 3               |
| /admin            | 3               |

This is worth flagging clearly. An earlier look at this site found the duplication **only** when entering on the bare domain, which redirects to `/home`. That is not the whole picture. Navigating between routes inside the app duplicates as well, which is why `/peptides`, `/cart` and `/checkout` all appear above.

## Why it happens

The route tracker runs again for a single navigation. A first render guard covers only the initial mount, so it does not protect against the component re-running on a client side route change.

## The fix

**Remember the last path that was tracked, and skip when it has not changed.**

```
const lastPath = useRef(null);

useEffect(() => {
  const path = location.pathname;
  if (lastPath.current === path) return;
  lastPath.current = path;
  whop.track("page", { path });
}, [location.pathname]);
```

Keying the guard on **the path** rather than on "have I run before" is the important part. It survives re-renders, redirects and client side navigation, and it still tracks correctly when a visitor genuinely leaves a page and later returns to it.

## How to verify it worked

Load the bare domain, then click through to two other pages. In the Whop events view you should see one page event per page actually visited. Before the fix, that sequence produces roughly four to five events for three pages.

Conversions are not affected by this one, so fixing it changes traffic numbers but not revenue numbers.

### 3. Smaller: your own admin traffic is being tracked

---

`/admin` and `/admin/orders` account for **35 page events** in the same ten day window. That is staff working in the back office being counted as site traffic.

It does not corrupt revenue, but it does inflate visits and it drags conversion rate down, because those sessions can never buy anything.

The fix is to skip tracking when the visitor is a logged in staff user, or when the path begins with `/admin`. The path check is the simpler of the two:

```
if (location.pathname.startsWith("/admin")) return;
```

### What is already correct, please do not change it

---

These were each verified working and should be left alone:

- **The pixel loads and initializes.** The loader is present site wide and the event queue drains rather than backing up.
- **The purchase payload shape is right.** Value, currency, event id, email and phone are all present and correctly typed.
- **Sales genuinely reach Whop.** The purchase request returns success and Whop mints a stored event id in response, which is the only real proof of receipt.
- **Purchases are scoped to the correct business.** Every event checked carried the right company id and no other.

### Summary for whoever schedules this

---

| # | Problem                                    | Impact                                     | Size of fix |
|---|--------------------------------------------|--------------------------------------------|-------------|
| 1 | Sale sent under two different event ids    | Revenue reads about 2x                     | One line    |
| 2 | Route tracker fires twice per navigation   | Traffic reads about 23% high               | A few lines |
| 3 | Admin browsing tracked as customer traffic | Visits inflated, conversion rate depressed | One line    |

Problem 1 is the one that matters most, because it is the one that makes a business number wrong rather than a traffic number.